



IN-CLASS
EXERCISES

SEMANTIC ANALYSIS 3 —TYPE CHECKING

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`sem-3-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip sem-3-exercises.zip
```

```
> cd sem-3-exercises
```

```
> ls
```

INTRODUCTION

Standard Types

- The class `Standard.java` declares symbols for standard types in the language:

```
public class Standard {  
  
    public static BasicType IntType;  
    public static BasicType CharType;  
    public static BasicType RealType;  
    public static BasicType BoolType;  
  
    ...  
}
```

Type Compatibility

- Sem. java declares functions for type compatibility in the language.

```

/*****
/* resultType(op,L,R) returns the resulting type when we apply the
/* operator op to the types L and R.
*****/
TypeSy resultType (int op, TypeSy L, TypeSy R) {
    return ...
}

/*****
/* isArithmeticType(T) returns true if T is a type you can
/* perform arithmetic on.
*****/
boolean isArithmeticType (TypeSy T) {
    return (T == Standard.IntType) || (T == Standard.RealType);
}
```

```

/*****
/* areCompatible(OP,L,R) returns true if you can apply the binary operator */
/* OP to variables of type L and R. */
*****/
boolean areCompatible(int op, TypeSy L, TypeSy R){
    return ...
}

/*****
/* areAssignmentCompatible(L,R) returns true if a value of type R */
/* can be assigned to a variable of type L. */
*****/
boolean areAssignmentCompatible(TypeSy L, TypeSy R){
    return ...
}

```


- These functions will make use of the basic type symbols in `Standard.java`. Here's an example of what that might look like:

```
/**
 * isArithmeticType(T) returns true if T is a type you can
 * perform arithmetic on.
 */
boolean isArithmeticType (TypeSy T) {
    return (T == Standard.IntType) || (T == Standard.RealType);
}
```

Use of Compatibility Functions

- In Sem. java we use these functions to check context conditions:

```
void assignStat(ASSIGN n) {
    n.expr.env = n.env;
    Symbol sy = n.env.lookup(n.ident);
    if (sy == null) {
        ERROR("Identifier not declared: " + n.ident);
    } else if (sy instanceof Constant) {
        ERROR("Cannot assign to a constant: " + n.ident);
    } else {
        expr(n.expr);

        if (! (areAssignmentCompatible(n.expr.type, sy.type))) {
            ERROR("Wrong assignment type: " + sy.type + "; " + n.expr.type);
        }
    }
}
```


Environments

- In Sem. java we pass environments around everywhere, so we can use them too look up symbols:

```
void binop(BINOP n) {  
    n.left.env = n.right.env = n.env;  
    expr(n.left);  
    expr(n.right);  
  
    ...  
}
```

```
void ident(IDENT n) {  
    Symbol sy = n.env.lookup(n.ident);  
    if (sy == null) {  
        ERROR("Not declared: " + n.ident);  
    } {  
        n.type = sy.type;  
    }  
}
```

TASK 1

Task 1 (a)

- Update the `resultType` function so that:
 - `integer OP integer` returns `integer`, where `OP` is `+`, `-`, `*`
 - `float OP float` returns `float`, where `OP` is `+`, `-`, `*`
 - `integer < integer` returns `boolean`
 - `float < float` returns `boolean`

```
/* **** */
/* resultType(op,L,R) returns the resulting type when we apply the */
/* operator op to the types L and R. */
/* **** */
TypeSy resultType (int op, TypeSy L, TypeSy R) {
    return ...
}
```


Task 1 (b)

- Test your code:

task1a.tny

```
BEGIN
  VAR    x : int;
  VAR    y : int;
  PRINT x + 1;
END
```

task1b.tny

```
BEGIN
  VAR    x : int;
  VAR    y : flt;
  PRINT x + y;
END
```

task1c.tny

```
BEGIN
  VAR    x : flt;
  VAR    y : flt;
  PRINTs x + 1.0;
END
```

> java Sem task1a.tny

> java Sem task1b.tny

[Semantic Error] Wrong binary type: int PLUS flt

> java Sem task1c.tny

Task 1 (c)

- Change `resultType` and `areCompatible` so that the type rules resemble those of the C language:
 - integer OP float returns float, where OP is +, -, *
 - float OP integer returns float, where OP is +, -, *

```
/* **** */
/* resultType(op,L,R) returns the resulting type when we apply the */
/* operator op to the types L and R. */
/* **** */
TypeSy resultType (int op, TypeSy L, TypeSy R) {
    return ...
}
/* **** */
/* areCompatible(OP,L,R) returns true if you can apply the binary operator */
/* OP to variables of type L and R. */
/* **** */
boolean areCompatible(int op, TypeSy L, TypeSy R){
    return ...;
}
```

Task 1 (d)

- Test your code:

task1a.tny

```
BEGIN
  VAR    x : int;
  VAR    y : int;
  PRINT x + 1;
END
```

task1b.tny

```
BEGIN
  VAR    x : int;
  VAR    y : flt;
  PRINT x + y;
END
```

task1c.tny

```
BEGIN
  VAR    x : flt;
  VAR    y : flt;
  PRINTs x + 1.0;
END
```

```
> java Sem task1a.tny
> java Sem task1b.tny
> java Sem task1c.tny
```


TASK 2

Task 2 (a)

- Update the `areAssignmentCompatible` function so that it encodes C-like assignment compatibility:
 - `integer := integer` is OK
 - `float := float` is OK
 - `float := integer` is OK
 - `integer := float` is Illegal

```
/* **** */
/* areAssignmentCompatible(L,R) returns true if a value of type R
/* can be assigned to a variable of type L.
/* **** */
boolean areAssignmentCompatible(TypeSy L, TypeSy R){
    return ...;
}
```

Task 2 (b)

- Test your code:

task2a.tny

```
BEGIN
  VAR    x : int;
  VAR    y : flt;
  x = y;
END
```

task2b.tny

```
BEGIN
  VAR    x : int;
  VAR    y : flt;
  y = x;
END
```

```
> java Sem task2a.tny
[Semantic Error] Wrong assignment type: int; flt
> java Sem task2b.tny
```


TASK 3 - ARRAYS

Arrays

- I have added one-dimensional arrays to Tiny.
- Here's the syntax of array declarations and index expressions:

array1.tny

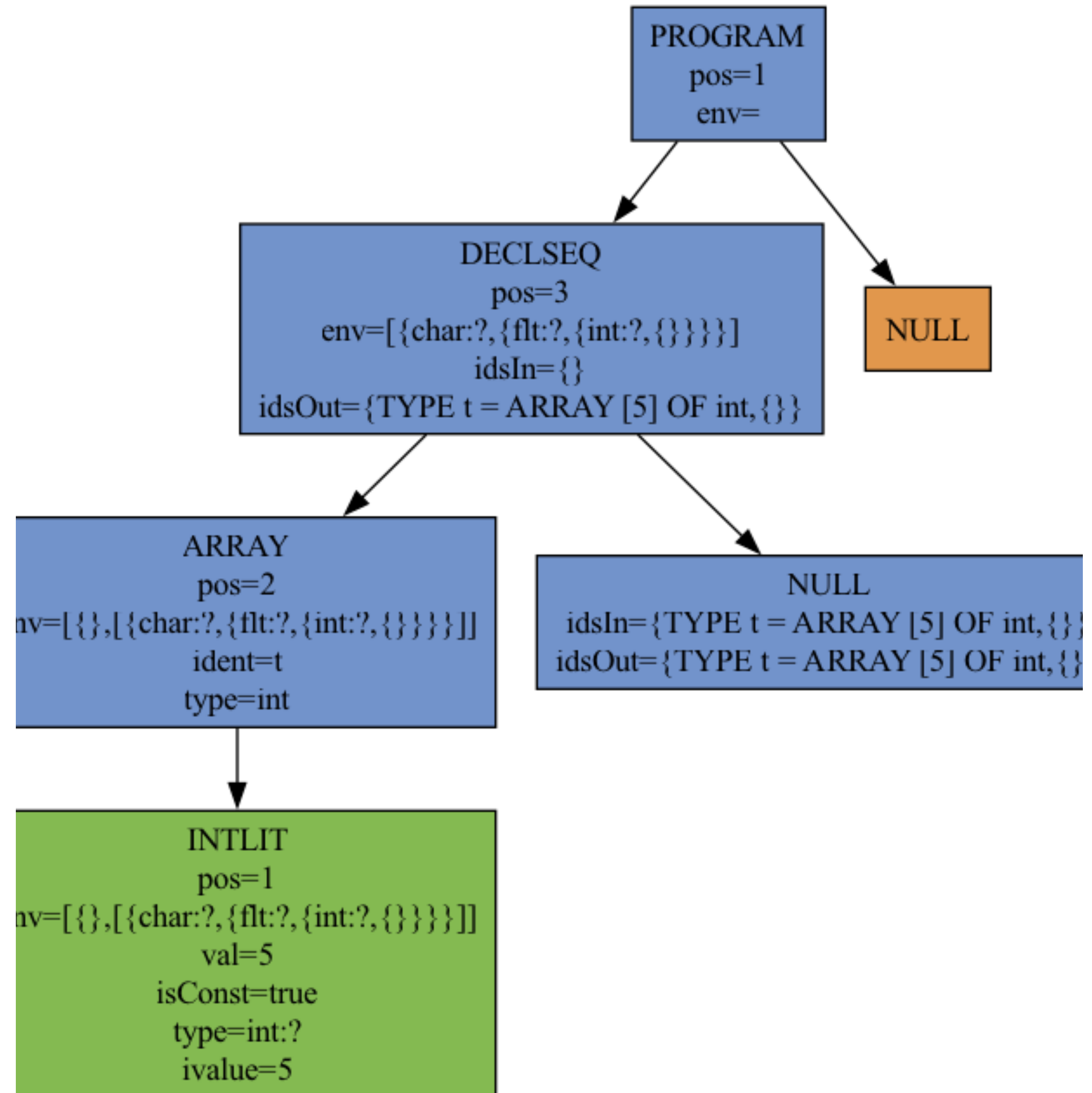
```
BEGIN
    TYPE t = ARRAY 5 OF int;
    VAR x : int;
    VAR a : t;
    x = a[5];
    a[5] = x;
    x = a[5] + a[0];
END
```

Array Declaration AST

- Here's what the AST looks like for an array declaration:

array0.tny

```
BEGIN
    TYPE t = ARRAY 5 OF int;
END
```

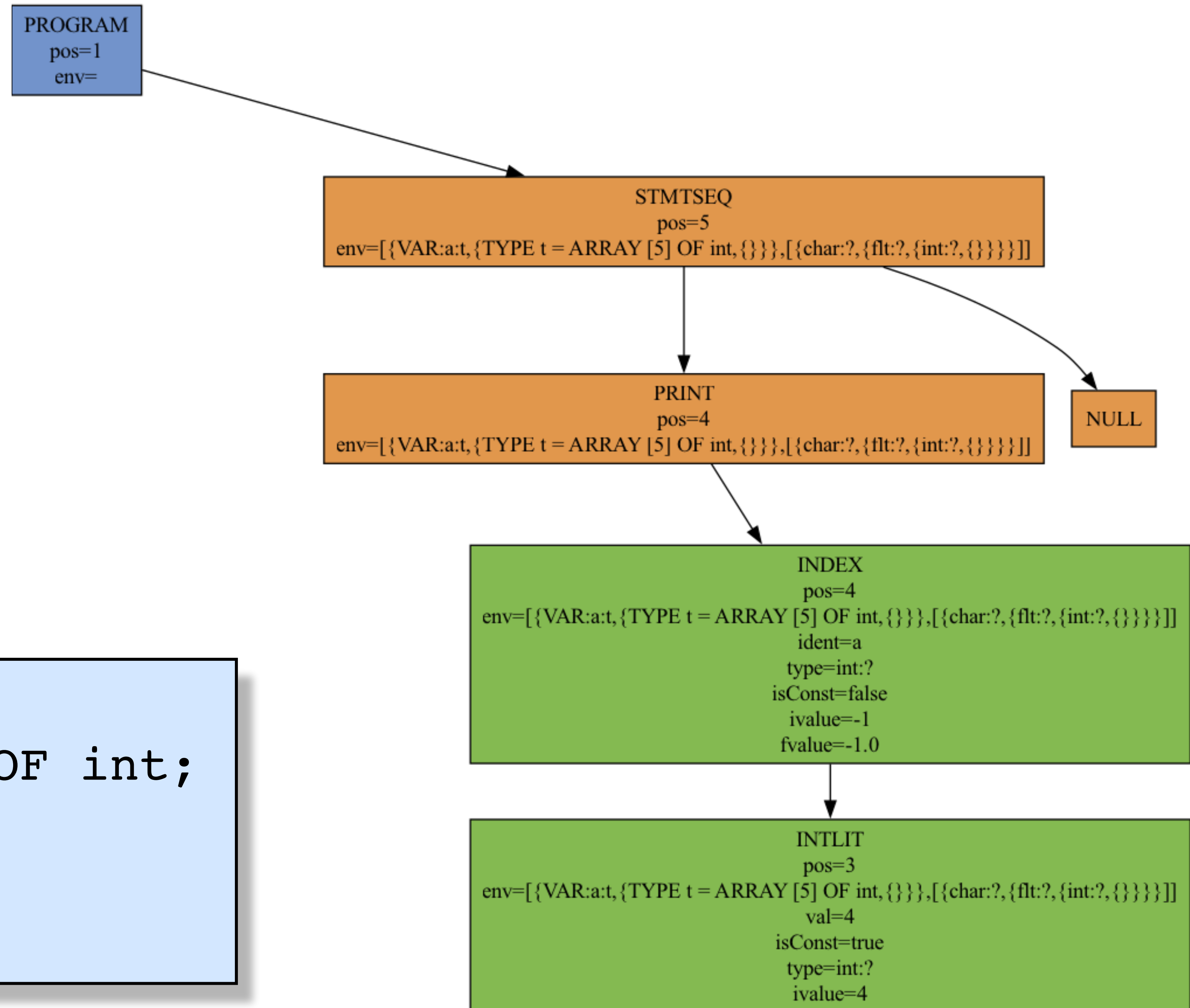


Array Index AST

- Here's what the AST looks like for an index expression:

array4.tny

```
BEGIN
  TYPE t = ARRAY 5 OF int;
  VAR a : t;
  PRINT a[4];
END
```



Index Context Conditions (a)

- There are 4 conditions that the semantic analyzer has to check for an index expression:

1. The identifier should be declared
2. The identifier should be a variable

array5.tny

```
BEGIN
    TYPE t = ARRAY 5 OF int;
    PRINT a[4];
END
```

array6.tny

```
BEGIN
    TYPE t = ARRAY 5 OF int;
    PRINT t[4];
END
```

> java Sem array5.tny

[Semantic Error] Identifier not declared: a

> java Sem array6.tny

[Semantic Error] Variable expected: t

Index Context Conditions (b)

3.The type of A should be an array type T

4.The index expression should be an integer

array7.tny

```
BEGIN
  VAR a:int;
  PRINT a[4];
END
```

array8.tny

```
BEGIN
  TYPE t = ARRAY 5 OF int;
  VAR a:t;
  PRINT a[42.42];
END
```

> java Sem array7.tny

[Semantic Error] Array type expected: a

> java Sem array8.tny

[Semantic Error] Array index type should be an integer: flt

New Java Classes

- `ArrayType.java` is the class that holds a symbol that's an array type:

```
public class ArrayType extends TypeSy {  
    public int Count = 0;  
    public TypeSy ElementType = null;  
}
```

- `INDEX.java` is the class that holds a AST node for an index expression:

```
public class INDEX extends EXPR {  
    public String ident;  
    public EXPR index;  
}
```


Task 3

- Edit the code for INDEX AST nodes in `Sem.java` to check the four context conditions.
- A skeleton code has been provided for you on the next page and in the file `task3-skeleton.txt`.

```

void index(INDEX n) {
    n.type = Standard.NoType;
    Symbol sy = n.env.lookup(n.ident);
    if ( ... ) {
        ERROR("Identifier not declared: " + n.ident);
    } else {
        if ( ... )) {
            ERROR("Variable expected: " + n.ident);
        } else {
            TypeSy arrType = ...;
            if ( ... )) {
                ERROR("Array type expected: " + n.ident);
            } else {
                Type check the index expression
                if ( ... ) {
                    ERROR("Array index type should be an integer: " + n.index.type.ident);
                } else {
                    n.isConst = false;
                    n.type = ...;
                }
            }
        }
    }
}

```




DEFINITIONS AND ALGORITHMS

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA